

Comparing Information Retrieval in AI-Powered and Traditional Search Systems

Shahab Kiyani and Sarang Kale

TABLE OF CONTENTS

- I. Introduction - pg. 3
- II. Literature Review - pg. 5
- III. System Design and Implementation - p. 18
- IV. Results - p.32
- VI. Challenges and Debugging - p. 40
- VII. Discussion - p. 48
- VIII. Future Work - p. 54
- IX. Bibliography - p. 56

1. Introduction

Search engines are among the most influential technologies of the modern era, acting as primary gateways to information, education, and decision making. They have fundamentally transformed how individuals access knowledge, conduct research, and navigate the digital landscape. What began as simple keyword-matching systems in the 1990s has evolved into sophisticated, AI-driven platforms capable of understanding natural language, generating contextual responses, and personalizing results based on user behavior. This evolution represents not just incremental technical improvements, but a completely new concept of what it means to search for information.

Traditional search engines, exemplified by early versions of Google, AltaVista, and Yahoo, operated on relatively straightforward computational principles. These systems indexed web content through automated crawlers, matched user queries to relevant documents through keyword analysis and statistical methods, and ranked results using algorithms like PageRank that leveraged the hyperlink structure of the web. The technical architecture underlying these systems included inverted indices, term frequency-inverse document frequency weighting, boolean query processing, and link analysis algorithms. These components represented elegant solutions to the core information retrieval problem. Given a massive corpus of documents, how can relevant materials be identified and ranked efficiently in response to user queries?

The integration of artificial intelligence, particularly large language models and natural language processing technologies, has totally altered this technical landscape. Modern AI-enhanced search engines like Google Search with AI Overviews, Bing Copilot, and Perplexity AI no longer simply return lists of links. Instead, they utilize neural networks trained

on vast collections of documents to interpret user intent, generate synthesized answers, engage in conversational exchanges, and provide tailored summaries that claim to pull relevant information from multiple sources. This architectural shift moves from document retrieval to answer generation, from ranking existing content to creating new text, and from transparent algorithmic processes to neural network decision-making where the reasoning behind results remains unclear.

This Honors Thesis investigates how the integration of artificial intelligence has reshaped search engine functionality, effectiveness, and user experience when compared to traditional, non-AI systems. Working collaboratively with Sarang Kale, who focuses on user perception and survey-based evaluation, my contribution centers on the technical implementation and system-level analysis that grounds our comparative study. A unique and critical component of this project is the construction of our own custom traditional search engine, which I am designing and implementing from the ground up. This engine development encompasses web crawling mechanisms, document indexing structures, ranking algorithms based on classical information retrieval techniques, and a functional user interface that allows for real-world query processing.

The development process reveals the technical challenges that shaped traditional search. These challenges include managing the computational complexity of indexing large amounts of documents, balancing precision and recall in retrieval algorithms, optimizing query response times, handling query ambiguity without natural language understanding, and designing ranking functions that produce relevant results without AI assistance. These hands-on insights inform our comparative evaluation framework and provide essential context for interpreting the differences between traditional and AI-enhanced approaches.

This custom engine will be evaluated alongside established AI integrated search platforms including Google Search with AI features, Bing Copilot, and Perplexity AI. We are using a two perspective framework that combines my technical performance analysis with Sarang's user perception studies. My work focuses on measurable system characteristics such as query processing speed, retrieval accuracy, ranking consistency, scalability constraints, and architectural transparency. Meanwhile, Sarang's complementary research examines how users perceive, trust, and interact with these different system types. Together, these dual perspectives provide comprehensive evidence-based insights into whether AI-powered search systems meaningfully improve information retrieval or introduce new risks and inefficiencies.

2. Literature Review

Research on information retrieval has grown around a stable technical goal and a constantly changing context. The goal is to retrieve and rank useful information from large collections under limited time and memory. The context has shifted from small collections of documents to a web-scale environment and now toward systems that mix retrieval with neural ranking and sometimes generation. The six sources selected for this thesis capture that transformation from classical web search architecture and indexing theory to probabilistic ranking, neural retrieval, transformer attention, and recent concerns about generative AI inside search. The literature provides both a technical blueprint for how traditional search engines work in practice and a set of arguments for why AI-driven systems represent an architectural change rather than a simple upgrade. This review emphasizes system components and measurable behavior because the

thesis evaluates search engines through comparative analysis of their performance, not just through abstract claims about intelligence.

Brin and Page provide a foundational systems view of web search that remains relevant today because it treats search as an end-to-end pipeline. They introduce Google as a prototype and state that it “makes heavy use of the structure present in hypertext” (Brin, 1). This line matters technically because it frames relevance as something more than textual matching; hyperlink structure becomes a signal that can be modeled, stored, and computed at scale. Their abstract also identifies the core engineering objectives of a web search engine in a single sentence: Google is “designed to crawl and index the Web efficiently and produce much more satisfying search results than existing systems” (Brin, 1). The wording ties together efficiency and satisfaction, which translates directly into system metrics like crawling speed, the cost of building the index, and the quality of the search results produced.

Brin and Page also establish scale as the main constraint of web retrieval. They write that search engines index “tens to hundreds of millions of web pages involving a comparable number of distinct terms” and answer “tens of millions of queries every day” (Brin, 1). This is not just historical context; it explains why particular data structures and distributed designs became the standard. Any retrieval algorithm that requires scanning documents at query time is excluded by these numbers. A second indication is that accuracy and efficiency are inseparable. Ranking must be good, but it must also meet latency budgets. This is one reason why Brin and Page discuss not just ranking, but also crawling, storage, and indexing as first-class problems. They frame engineering as central to retrieval research, stating that “to engineer a search engine is a challenging task” (Brin, 1). The technical contribution here is that engineering difficulties shape the retrieval model that is achievable. A ranking function that is theoretically good but costly to

compute will not survive in web-scale conditions. Their approach places key structural choices at the center of evaluation, which aligns with this thesis, as it involves building a traditional search engine rather than treating traditional retrieval as a purely theoretical baseline. A key idea in their paper is that ranking on the web cannot rely only on one page's text. PageRank gives authority through link structure, and Brin and Page explain that the citation graph provides "an objective measure of its citation importance that corresponds well with people's subjective idea of importance" (Brin, 4). This captures why PageRank became influential: it produces a numerical signal that can be computed offline and used at query time to influence ordering. It also represents a specific kind of transparency—while the full web graph is complicated, the idea that links give authority is easier to understand compared to many neural network processes. In a traditional search engine, authority is a stored feature with a clear source, even if it's calculated using large-scale math. The bigger perspective offered by Brin and Page is that search quality emerges from the integration of components. They describe Google as "a complete architecture for gathering web pages, indexing them, and performing search queries over them" (Brin, 15). This framing is crucial for this thesis because the research questions target real-world behavior. It's not enough to ask whether a model is accurate in isolation; the system must crawl, preprocess, store, retrieve, and rank under time and memory constraints. Traditional keyword-based search can be understood most clearly by tracing what happens to a document before a user ever types a query and then what happens when the query arrives. Brin and Page emphasize that a web search engine is not only a ranking algorithm but a full system for acquiring, organizing, and serving information at scale, describing Google as "a complete architecture for gathering web pages, indexing them, and performing search queries over them" (Brin, 15). This definition highlights an implementation-level reality that motivates the present

thesis. Retrieval behavior is shaped by pipeline design decisions such as what is crawled, what is stored, how it is normalized, and how quickly the index can be accessed under load.

Manning, Raghavan, and Schütze provide the most comprehensive grounding for the internal mechanics of traditional retrieval pipelines. Their textbook treats retrieval as a sequence of concrete computational tasks and connects the target for modern search engines directly to performance measurement. They note that major systems provide results “within subsecond response times for hundreds of millions of searches a day over billions of web pages” (Manning, 32). This statement ties ranking to systems engineering, where subsecond latency is not merely a preference but a requirement driven by user interaction and competitive pressure. Achieving this implies the use of caching, compression, careful memory layout, and an index that supports fast intersection and scoring. The central data structure enabling this is the inverted index. Manning and colleagues explain that “the dictionary is commonly kept in memory, with pointers to each posting's list, which is stored on disk” (Manning, 44). This design decision becomes evident in implementation: keeping the dictionary in memory accelerates term lookup, while storing postings on disk reduces RAM requirements. The tradeoff between these choices is influenced by hardware limits and index size. In a student-built search engine, similar principles can be seen, such as keeping term-to-posting mappings in memory for a smaller corpus or relying on local storage for larger crawls. Manning’s emphasis on compression and storage is important for understanding why traditional systems behave predictably. Terms map to postings lists, and functions operate on counts and lengths that can be computed directly. This predictability supports reproducibility, meaning the same query over the same index yields the same ranked list unless randomness is introduced intentionally. When comparing against AI-integrated systems, predictability and stability become measurable properties. If two runs with the same query

produce different outputs due to sampling or model updates, the system's behavior becomes harder to evaluate using classical retrieval methodologies.

In classical systems, the first major implementation step is converting raw text into an indexable representation. Manning and colleagues explain how the inverted index separates the vocabulary structure from postings, noting that “the dictionary is commonly kept in memory, with pointers to each postings list, which is stored on disk” (Manning, 44). This decision predicts multiple measurable performance properties: keeping the dictionary in memory reduces the time to locate postings for a query term, while disk-based postings introduce latency that depends on compression, caching, and access patterns. Query response time, therefore, becomes closely tied to data locality and how aggressively postings lists are compressed. In a built system, changing index layout can impact speed even if the ranking function remains identical. Indexing also determines which matching signals a system can exploit. If the index stores only document identifiers per term, the system supports basic term presence checks and straightforward scoring. If it stores term positions, it can support phrase matching and proximity features. Manning's description that “in a non-positional inverted index, a posting is just a document ID” (Manning, 44) provides a clean baseline for understanding this tradeoff. Non-positional indexing is simpler and smaller, improving speed and storage, while positional indexing increases index size but allows richer retrieval features. These choices affect ranking quality on queries where word order or phrases carry meaning. The present thesis treats this as a design variable in the traditional engine, which will be compared against AI systems that interpret phrases through learned representations rather than explicit positional lists. Ranking in traditional engines follows naturally from index statistics. Ranked retrieval replaced strict Boolean matching because users benefit from ordering rather than a flat set. Manning and colleagues connect modern search

behavior to this need for ranking at scale, observing that major systems deliver “high-quality results within subsecond response times” (Manning, 32). The key point is that ranking must be computed quickly, shaping which signals can be used. Scoring functions that depend on per-document term statistics are feasible because they are retrieved from postings, but scoring functions that require heavy computation over long document text at query time become too slow.

Probabilistic ranking models provide a principled approach for the ordering problem. Robertson and Zaragoza describe the key operation as “ranking documents in relation to a given query in descending order of probability of relevance” (Robertson, 4). This framing matters for implementation because it clarifies what the system is approximating. The engine computes a score intended to correlate with relevance probability and then sorts documents by that score. It also clarifies why the top of the ranking matters more than exhaustive classification. The system is judged by how well it places good documents early, which fits both user behavior and evaluation metrics. BM25 remains central because it operationalizes probabilistic relevance with statistics that are index-friendly. Robertson and Zaragoza describe BM25 as “one of the most successful text retrieval algorithms” (Robertson, 3). Its success is tied to feasibility. It uses term frequency and inverse document frequency, plus document length normalization, all of which can be computed from postings and stored document lengths. In a built system, BM25 can be implemented with fast operations over postings lists, making it suitable for low-latency search. This is a major reason BM25 remains a strong baseline in this thesis. It reflects what real systems can do quickly and predictably. Robertson and Zaragoza formalize the probabilistic reasoning behind traditional ranking models through the Probabilistic Relevance Framework, which they describe as a basis for estimating relevance probabilities. They define the key retrieval principle

as “ranking documents in relation to a given query in descending order of probability of relevance” (Robertson, 4). This matters because it expresses ranking as an ordering problem rather than a classification problem. The system does not need to decide whether a document is relevant in an absolute sense; it must order documents so that highly relevant ones appear early. That aligns directly with metrics such as precision and NDCG, which evaluate ordering quality in the top ranks. The framework led to the development of BM25, which Robertson and Zaragoza describe as “one of the most successful text retrieval algorithms” (Robertson, 3). The relevance for this thesis is twofold: First, BM25 provides a strong baseline that remains competitive in many settings, so a traditional engine can be evaluated using a ranking function with known properties. Second, BM25 is interpretable. Its scoring depends on term frequency, inverse document frequency, document length normalization, and tunable parameters. These elements map to measurable index statistics, which means that differences in ranking can often be traced back to differences in term distributions or parameter choices. That traceability becomes an important contrast when AI-driven systems rely on internal representations that are not directly tied to simple counts.

Mitra and Craswell explain why traditional methods that rely on exact word matching have limitations, which is why neural retrieval systems are needed. One major issue is vocabulary mismatch: users often express their needs in one way while documents use different words to convey the same ideas. A system that only matches words exactly might miss relevant documents. Mitra and Craswell highlight the need for neural retrieval by explaining that it’s important to understand the meaning of text beyond just individual words. They note that “learning latent representation of text is important for dealing with vocabulary mismatch, but exact matching is also important to deal with rare terms and intents” (Mitra, 13). This is key

because it rejects the idea that neural methods can fully replace word-matching systems. Instead, it suggests that a good search system needs to balance meaning-based matching with exact word matching, especially when dealing with rare terms, names, or specific phrases. This balance directly influences how search systems are built. Even without new data sources, Mitra and Craswell's point explains why hybrid systems are used. Exact match components are helpful for handling rare terms or very specific queries, while semantic components help when the query and document use different words to describe the same thing. From an implementation perspective, this means systems can be evaluated based on how they handle these different types of queries. For example, a traditional search engine might do well with rare terms but struggle with paraphrasing. A neural system might handle paraphrasing better but have trouble with rare terms or unfamiliar entities. This understanding allows the thesis to design tests that measure how systems perform in these specific scenarios, linking the Mitra and Craswell literature to real-world system behavior. Traditional engines also expose a clear separation between retrieval and presentation. The output is a ranked list of documents, often with titles, snippets, and links, and the user is responsible for opening sources. That separation shapes both trust and evaluation. Because each ranked item corresponds to a specific document, correctness can be assessed by judging whether that document satisfies the information needed. If the ranking is wrong, the failure mode is typically omission, meaning the system failed to retrieve or failed to rank highly a relevant document. This kind of failure is diagnosable through index inspection, query term analysis, and score breakdowns. Neural retrieval changes this picture by introducing learned representations that aim to capture meaning beyond exact overlap. Mitra and Craswell emphasize why this direction emerged, stating that "learning latent representation of text is important for dealing with vocabulary mismatch" (Mitra, 13). This line points to a concrete advantage: A

neural system can retrieve a document that does not share surface words with the query if the embedding space places them nearby. In classical engines, that retrieval would require manual query expansion or synonym resources. Neural models attempt to learn these relationships directly from data. Mitra and Craswell also warn that semantic matching is not enough. They write that “exact matching is also important to deal with rare terms and intents” (Mitra, 13). This is an implementation-relevant constraint. Many queries contain names, codes, niche entities, or newly emerging terms where training data coverage is limited. In those cases, lexical matching provides robustness. This statement predicts why hybrid pipelines are common in practice and provides a technical hypothesis that can be tested in the thesis. Traditional engines may perform strongly on rare term queries because inverted indices directly support exact matching, while AI-driven systems may struggle if embeddings fail to represent the rare term reliably.

The shift toward neural methods is closely tied to progress in representation learning and sequence modeling, with Vaswani and his team playing a key role. They introduced a model architecture that removed recurrence and instead focused on attention mechanisms. They describe it as “a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely” (Vaswani, 1). This is important for retrieval research because it paved the way for large language models, which are now vital to ranking, query understanding, and answer generation. The Transformer serves as a bridge between early neural ranking techniques and today’s more advanced generative systems. The paper also explains attention in a way that clarifies how it scales and changes the way models understand relationships in text. Vaswani and his colleagues define attention as “an attention function can be described as mapping a query and a set of key-value pairs to an output” (Vaswani, 3). This is especially important for search, as retrieval is essentially about matching

queries to documents. Traditional retrieval works by mapping a query to documents using term-based indexes and scoring. In contrast, attention maps a query representation to relevant information through weighted combinations. Both methods aim to prioritize relevant signals, but they use different computational approaches. Traditional retrieval relies on sparse term-document relations stored in an index, while attention uses dense vector calculations performed during inference. This fundamental difference can result in performance differences in areas like speed and hardware needs.

Transformer architectures strengthened neural methods by improving language modeling and representation learning. Vaswani and colleagues describe the Transformer as “based solely on attention mechanisms” (Vaswani, 1). From a search perspective, attention changes what it means to represent a query or a passage. Instead of relying on bag-of-words statistics, models can encode context-sensitive meaning where the representation of a word depends on surrounding words. This is one reason AI systems can handle conversational queries better than classical engines. At the same time, attention introduces computational costs that do not scale like postings list operations. Dense vector operations and attention-based encoding require more compute per query, which affects latency, throughput, and hardware requirements. The mechanics of attention also clarify why these models can capture long-range dependencies in text. Vaswani and colleagues explain that “an attention function can be described as mapping a query and a set of key-value pairs to an output” (Vaswani, 3). This view resembles retrieval in an abstract sense because both involve scoring relevance between a query and candidate information. The difference is where the scoring happens and what is being scored. Traditional retrieval scores documents using precomputed statistics accessed through an index. Attention

computes dynamic weights over representations during inference. The outcome is greater flexibility and better semantic generalization, but also a heavier computation footprint.

The rise of generative AI inside search introduces a separate but connected set of concerns addressed by Hersh. He frames recent changes as a disruption of a previously mature technology. He writes that information retrieval systems “had been relatively mature applications until late 2022” and that this stability was “upended by the emergence of generally available generative artificial intelligence chatbots” (Hersh, 2). This is a technical claim as much as a social claim, implying that evaluation practices, system goals, and user expectations are rapidly changing. If search shifts from returning ranked documents to producing synthesized answers, core retrieval assumptions about output objects and relevance judgments become less direct. Hersh highlights a key issue that arises when generative models are combined with retrieval: the problem of traceability. He points out that users have “no idea what actual source text led the generative model to output the specific text” (Hersh, 4). This is significant because it shows a lack of attribution and where the source came from. In traditional retrieval systems, each ranked result is a document with a clear URL or identifier. However, in generative systems, the output might combine sources, leave out citations, or present content that cannot be traced back to a single origin. This makes debugging harder, and it also complicates evaluation, as it becomes difficult to determine whether an incorrect result is due to retrieval issues, ranking errors, or summarization problems.

These architectural differences become even more pronounced when search systems shift from retrieval to generation. Hersh emphasizes that information retrieval remains widely used, and the arrival of generative chatbots disrupted a mature landscape, noting that retrieval systems “had been relatively mature applications until late 2022” (Hersh, 2). This matters technically because

maturity implies established evaluation methods, stable architectures, and predictable failure modes. Generative integration introduces new components and, therefore, new uncertainty. The system is no longer limited to retrieving and ranking documents; it may synthesize a response and compress multiple documents into a single narrative. Hersh identifies the technical risk created by this change through the provenance problem. He notes that the user has “no idea what actual source text led the generative model to output the specific text” (Hersh, 4). In implementation terms, this is a traceability gap. If a generated sentence is incorrect, the system designer may not be able to locate whether the error came from retrieval, from ranking, from summarization, or from model hallucination. Traditional engines allow direct mapping from output to document, but generative systems can break that mapping unless they provide strong citation alignment and faithful grounding.

This shift also changes how performance should be measured. Classical systems can be evaluated by how well they order documents by relevance and by how quickly they return results. AI-integrated systems still face latency constraints, but the output object is sometimes a generated answer rather than a ranked list. Errors may be more severe because a single confident answer can mislead more than a low-ranked link. The difference between omission and fabrication becomes central. In a traditional engine, a user may fail to find the right source quickly. In a generative system, a user may accept an incorrect statement that looks authoritative. The present thesis treats these differences as measurable technical tradeoffs rather than as abstract claims about intelligence. Traditional retrieval can be analyzed by index behavior, ranking stability, and response performance grounded in inverted index design and probabilistic scoring. Neural retrieval can be analyzed by semantic generalization benefits and by cost, interpretability, and robustness challenges tied to dense representations and attention-based

computation. AI-integrated search that includes generation can be analyzed by provenance, traceability, and the reliability of grounding relative to the transparency of document-based results. This framing sets up the comparative technical evaluation that follows. It motivates designing tests that isolate exact match versus semantic match behavior, evaluating response time under comparable conditions, examining ranking consistency across repeated queries, and documenting failure modes where systems either omit relevant documents or produce ungrounded statements.

Across these sources, a clear technical narrative emerges. Traditional web search evolved through pipeline engineering, scalable indexing, and probabilistic ranking. Brin and Page focused on architecture and web-specific signals, while Manning detailed the data structures and efficiency needs for large-scale retrieval. Robertson and Zaragoza provided the probabilistic foundation for ranking, explaining why BM25 remains a reliable baseline. Mitra and Craswell identified the lexical limits that led to the development of representation learning, while also clarifying the ongoing need for exact matching in neural systems. Vaswani's work provided the breakthrough architecture that enabled modern language models to scale. Hersh discussed how generative models changed the search landscape, introducing new behaviors and risks related to traceability. This narrative directly ties to this thesis's research questions at a technical level. The question of how traditional engines retrieve, rank, and present information is addressed by Brin and Page's pipeline model and Manning's analysis of inverted indices, compression, and ranked retrieval under latency constraints. The question of what changes are brought by AI-driven systems is explained by the shift from term-based scoring to learned representations and attention-based computation, as well as the move from ranked documents to synthesized outputs, as discussed by Hersh. The question of where AI improves effectiveness is linked to vocabulary

mismatch and semantic generalization, as described by Mitra and supported by Transformer attention. Lastly, the question of new limitations aligns with the computational cost, interpretability, and traceability issues highlighted by Hersh.

This literature review establishes the foundational principles of traditional and AI-driven search systems, setting the stage for the comparative technical evaluation that follows, which will assess how these systems perform in real-world scenarios.

3. System Design & Implementation

3.1 Dataset Selection: BEIR Natural Questions

The system uses the Natural Questions dataset from the BEIR benchmark suite. BEIR was selected because it provides standardized information retrieval datasets that include a document corpus, user queries, and relevance judgments. This made it useful for a thesis focused on evaluating retrieval behavior because the system could compare retrieved documents against known relevant documents instead of relying only on subjective impressions.

The Natural Questions dataset was especially useful because it is based on real information-seeking queries and Wikipedia-style documents. Since this project compares traditional and AI-supported search behavior, a Wikipedia-based corpus provided a practical middle ground: the data was large enough to make retrieval meaningful, but structured enough to

support controlled testing. In the implementation, the dataset was loaded from the local ``datasets/nq`` folder using BEIR's ``GenericDataLoader``, which returned three main components: the corpus, the queries, and the qrels relevance judgments. The indexing script prints the number of loaded corpus documents, queries, and qrels after loading the test split, which made it easier to verify that the dataset had been loaded correctly before indexing began.

The corpus contains document IDs mapped to document content, while the qrels file maps query IDs to relevant document IDs. This structure was important because it allowed the system to connect three separate pieces of information: the user's query, the documents retrieved by the search engine, and the documents labeled as relevant by the benchmark. Without this structure, the system could still return results, but it would be much harder to measure whether those results were correct.

3.2 Document Preparation and Stable ID Mapping

After loading the BEIR dataset, the system creates a stable list of document IDs from the corpus. This step matters because the embedding index returns numerical positions, while the BEIR dataset identifies documents using original document IDs. To preserve this mapping, the implementation stores the corpus keys in a list called ``doc_ids``. The order of this list is treated as stable, meaning that index position ``0``, ``1``, ``2``, and so on can later be mapped back to the correct BEIR document ID.

Each document is then converted into a searchable text string by combining its title and body text. The implementation extracts the title and text fields from each BEIR document, handles missing values by replacing them with empty strings, and joins them into a single document string. This design choice improves retrieval because titles often contain high-value terms, while body text provides additional context. The indexing script also prints progress every 10,000 documents, which was useful during large indexing runs because embedding generation can take a significant amount of time.

This step represents an important implementation decision. A search system does not retrieve from “documents” in an abstract sense; it retrieves from whatever representation of those documents the developer chooses to index. If only body text were indexed, short titles containing important entity names might be ignored. If only titles were indexed, the system would miss broader contextual information. Combining title and text created a simple but effective representation for retrieval.

3.3 Embedding Generation

Instead of using only a purely lexical method such as BM25, this proof-of-concept system uses dense vector embeddings through the `txtai` library. The embedding model used was `sentence-transformers/all-MiniLM-L6-v2`, a compact sentence-transformer model designed to

represent text semantically. The indexing script initializes a `txtai.Embeddings` object with this model and then calls `embeddings.index(documents)` to convert the prepared document strings into searchable vector representations.

This design choice reflects one of the central themes of the thesis: modern search increasingly depends on semantic representation rather than exact keyword matching alone. A lexical search engine depends heavily on whether query terms appear directly inside a document. In contrast, embedding-based retrieval attempts to represent queries and documents in a shared vector space, where semantically related texts can be close even when they do not use identical words. This allowed the system to behave more like a modern neural retrieval engine while still remaining controlled and inspectable.

At the same time, using embeddings introduced important tradeoffs. Embedding generation is computationally expensive compared to basic keyword indexing. It requires running every document through a neural model, which can take substantial time for a large corpus. However, once the embeddings are generated, retrieval becomes faster because the system can load the saved index rather than recomputing document vectors each time the application starts.

3.4 Saving and Reloading the Embedding Index

A major implementation goal was to avoid regenerating embeddings every time the system ran. To solve this, the indexing script saves the completed embedding index to a local folder called ``nq_embeddings``. It also saves the stable document ID list into a pickle file called ``nq_doc_ids.pkl``.

This persistence step was essential for practical development. Without saving the embeddings, every test run would require reloading the dataset and rebuilding the entire vector index. That would make experimentation slow and inefficient. By saving both the embeddings and the document ID mapping, the system separates the expensive offline indexing phase from the faster online search phase.

In other words, the project uses a two-stage architecture:

1. Offline indexing stage: Load the BEIR corpus, prepare documents, generate embeddings, and save the index.
2. ‘Online’ search stage: Load the saved embeddings, accept user queries, retrieve documents, generate answers, and compute metrics.

This mirrors the structure of real search systems. Production search engines usually do not build their index at query time. They perform crawling, processing, indexing, and ranking preparation ahead of time so that user queries can be answered quickly.

3.5 Streamlit Search Interface

The user-facing application was built with Streamlit. The main application loads the BEIR corpus, queries, qrels, saved document IDs, and saved embedding index inside a cached resource function. Streamlit's `@st.cache_resource` decorator is used so that these objects are not repeatedly reloaded on every interaction.

This was important because loading embeddings and datasets repeatedly would make the interface slow and frustrating to use. Caching helped create a smoother search experience and made the system more practical for repeated testing.

The interface itself begins with a title, 'Search Engine - BEIR NQ', and a text input box where users can enter a query. When the search button is clicked, the system sends the query to the embedding index and retrieves the top 20 results.

Each result returned by the embedding search includes an index position and a similarity score. The system maps the index position back to the original BEIR document ID using the saved `'doc_ids'` list. It then retrieves the corresponding document from the corpus and extracts the title

and text. To make the output more realistic, the system also generates a Wikipedia-style URL by converting the document title into a Wikipedia link format.

This design allowed the interface to resemble a real search engine results page. Instead of only showing document IDs and scores, the system displayed ranked results with titles, links, snippets, and retrieval scores.

3.6 Query Normalization and Matching to BEIR Queries

To compute evaluation metrics, the system needed to determine whether the user’s input matched one of the official BEIR queries. Since user input may vary in capitalization, punctuation, or spacing, the implementation includes a query normalization function. This function lowercases the text, removes punctuation, strips extra whitespace, and returns a simplified version of the query.

During data loading, the system creates a dictionary mapping normalized query text to its original BEIR query ID. This allows the application to match a typed query to a benchmark query when possible.

This step was necessary because the qrels relevance judgments are organized by query ID, not by raw query text. If the system can match the user’s query to a BEIR query ID, it can retrieve the relevant document IDs and compute formal metrics. If the query does not match a known BEIR query, the system can still retrieve and display results, but it cannot compute benchmark-based evaluation scores.

3.7 Retrieval Evaluation Metrics

The system computes several information retrieval metrics, including Precision@5, Recall@5, F1@5, first relevant rank, and nDCG@5. The `compute_ir_metrics` function compares the top retrieved documents against the set of relevant documents from qrels. It counts true positives, then calculates precision, recall, and F1.

Precision@5 measures how many of the top five retrieved results are relevant. Recall@5 measures how many of the total relevant documents were retrieved in the top five. F1@5 balances precision and recall into a single score. These metrics were useful because they allowed the system to quantify retrieval quality rather than relying only on visual inspection.

The system also calculates the first relevant rank, which identifies the first position where a relevant document appears in the retrieved list. If no relevant document appears, the function returns `'None'`.

Finally, the system computes nDCG@5. This metric accounts not just for whether relevant documents appear, but where they appear in the ranking. The DCG function adds relevance scores with a logarithmic discount based on rank position, meaning relevant documents near the top contribute more than relevant documents lower in the ranking. The nDCG function then compares the actual ranking to an ideal ranking.

This was important because search quality is not only about retrieving relevant documents somewhere in the list. In practical search behavior, users are much more likely to inspect the first

few results. A system that ranks a relevant document first is more useful than a system that places the same document at rank 20.

3.8 AI Overview Generation

In addition to displaying retrieved documents, the system includes an AI overview feature. This feature uses the top retrieved documents as context and sends them to a local LLM through Ollama. The function first builds a structured context from the retrieved documents, including each document's title and text.

The AI answer generation function then creates a prompt instructing the model to act as an AI search overview system. The prompt tells the model to generate a direct answer, include bullet points, remain neutral and concise, avoid mentioning the retrieval process, and use light inline citations such as `[1]` and `[2]`. Importantly, the prompt instructs the model to use only the provided documents.

This AI overview layer allowed the project to explore the difference between document retrieval and answer generation. The retrieval system returns ranked sources, while the AI overview converts those sources into a synthesized answer. This distinction is central to the thesis because modern AI search systems often blur the line between finding information and generating information.

The use of Ollama also made the system more practical because it allowed local generation through the `llama3.2` model. This reduced dependence on external API calls and helped avoid issues such as rate limits, quota restrictions, or inconsistent access to commercial LLM services.

3.9 Source Deduplication and Result Display

After generating the AI overview, the application displays sources from the top retrieved documents. To avoid showing duplicate links, the system maintains a set of already-seen URLs and only adds a source if its URL has not appeared before.

This was a small but meaningful design choice. During testing, retrieval results sometimes included documents with similar or repeated titles. Deduplicating sources made the interface cleaner and closer to what users expect from a modern search results page. However, it also revealed a deeper evaluation issue: visual deduplication for user experience can conflict with formal retrieval evaluation. This deduplication would also impact evaluation metrics since relevant documents were sometimes hidden. Given this, we resulted in including duplicate url's. A duplicate-looking result may still have a distinct BEIR document ID, and removing or hiding it can change how the system appears to perform.

The final interface displays three major pieces of information: the AI overview, the evaluation metrics, and the ranked search results. The metrics are shown using Streamlit metric cards for Precision@5, Recall@5, F1@5, and nDCG@5. The search results are displayed with rank number, title, URL, score, and a short text snippet.

This combined interface was valuable because it allowed both qualitative and quantitative evaluation in one place. A user could read the AI-generated summary, inspect the supporting sources, and view formal retrieval scores for benchmark queries.

3.10 Dataset Analysis Script

In addition to the main search system, a separate analysis script was written to inspect the qrels relevance structure. This script loads the same BEIR NQ dataset and counts the number of relevant documents for each query. It then sorts queries by the number of relevant documents and prints the top queries with the most relevant documents.

The script also calculates dataset-level statistics, including the maximum number of relevant documents for a query, the average number of relevant documents, and the number of queries with only one relevant document.

This analysis mattered because retrieval metrics behave differently depending on how many relevant documents exist for each query. For example, Recall@5 is much harder to maximize if a query has many relevant documents, while a query with only one relevant document creates a more binary evaluation problem. If the system retrieves that one document, recall is high; if it misses it, recall drops to zero. Understanding the qrels distribution helped interpret evaluation results more carefully.

3.11 Summary of Implementation

Overall, the system was designed as a controlled proof-of-concept search environment. The BEIR Natural Questions dataset provided a standardized corpus, queries, and relevance judgments. The indexing script generated dense vector embeddings using ``sentence-transformers/all-MiniLM-L6-v2`` and saved both the embedding index and document ID mapping for efficient reuse. The Streamlit application loaded the saved index, accepted user queries, retrieved ranked documents, generated an AI overview using a local LLM, displayed source links, and calculated formal information retrieval metrics when the query matched the benchmark.

This implementation process revealed that search engines are not defined by a single algorithm. They are pipelines made up of dataset decisions, representation choices, indexing strategies, retrieval methods, evaluation rules, and interface design. Each layer affects the final user experience and the measurable performance of the system. For this reason, building the system was not only a technical task, but also a research method. It made the tradeoffs between retrieval, ranking, generation, speed, transparency, and usability visible in a way that purely theoretical comparison could not.

4. Evaluation Methodology

4.1 Information Retrieval Metrics

To evaluate retrieval performance, this study uses standard information retrieval metrics, including Precision@k, Recall@k, F1@k, and normalized Discounted Cumulative Gain (nDCG@k). These metrics are widely used in retrieval research because they provide measurable insights into how effectively a system retrieves and ranks relevant documents.

4.2 Experimental Setup

The evaluation was conducted using the BEIR Natural Questions (NQ) dataset, which provides a standardized benchmark for information retrieval. The dataset includes a corpus of documents, a set of queries, and relevance judgments (qrels) that identify which documents are considered relevant for each query. Queries were selected from the BEIR dataset to ensure that each evaluation scenario had a corresponding set of relevance judgments. The system retrieves the top documents for each query using embedding-based similarity search. Evaluation metrics are then computed on the top 5 results to reflect realistic user behavior, as users typically focus on the highest-ranked results. This cutoff was chosen to balance computational efficiency with practical relevance. In addition to quantitative evaluation, the system generates an AI-based overview using the top retrieved documents as context. This allows for comparison between traditional ranked retrieval outputs and AI-generated summaries based on the same underlying documents. By keeping the document pool consistent, the evaluation isolates the difference between retrieval and generation rather than differences in data sources.

4.3 Survey-Based Evaluation Methodology

In addition to quantitative metrics, this study incorporates a survey-based evaluation component to examine perceived usefulness, clarity, and trustworthiness of search outputs.

Originally, this component was intended to involve external human participants. However, due to the need for Institutional Review Board (IRB) approval for human-subject research, and the absence of approval within the project timeline, a traditional survey study could not be conducted. To address this limitation while maintaining the exploratory goals of the thesis, an alternative methodology was adopted. A total of 150 survey responses were generated across three sources:

50 responses from the author (Shahab Kiyani)

50 responses from the project collaborator (Sarang Kale)

50 responses generated using large language models (LLMs)

These responses were used to simulate user evaluations of search results produced by both the custom search system and AI-enhanced search platforms.

The inclusion of LLM-generated responses introduces an additional layer of analysis, allowing comparison between human and machine-generated judgments. While these responses do not replace real user feedback, they provide insight into how evaluation criteria may differ across evaluators and help identify patterns in perceived relevance and usefulness.

5. Results

Overall System Preference

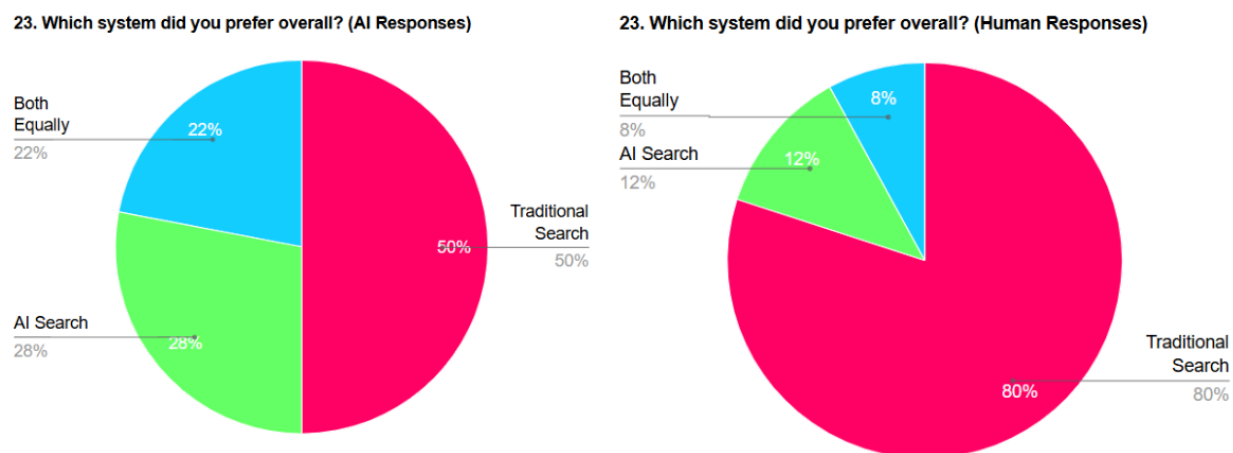


Figure 1: Human and AI responses comparing overall preference between traditional and AI-enhanced search engines.

One of the clearest trends observed across the dataset was a strong overall preference for traditional search systems. Approximately 70% of responses favored traditional search, while a substantially smaller percentage selected AI-enhanced systems or indicated no preference between the two. This finding is significant because it contrasts with the growing visibility and popularity of AI-powered search platforms. Although AI systems introduce features such as

direct answer generation and summarization, users still demonstrated a stronger preference for systems that present information through structured lists of sources. Figure 1 illustrates this overall preference distribution between traditional and AI-enhanced search systems.

A closer examination of the responses reveals several factors contributing to this preference. Many participants emphasized that traditional search engines allow users to compare multiple sources, inspect different perspectives, and independently evaluate credibility. This process contributes to a stronger sense of transparency because users can clearly identify where information originates. In contrast, AI-generated search results often provide a single synthesized response, reducing visibility into the retrieval and reasoning process. Even when citations are included, users may remain uncertain about how the response was constructed or which sources contributed most heavily to the answer.

These findings align closely with the technical observations described elsewhere in this thesis. While the custom search system is capable of retrieving relevant documents and generating AI-based summaries, user perception is influenced not only by retrieval quality but also by presentation style. From a system-level perspective, retrieval may function effectively; however, from a user perspective, transparency and control remain central components of trust.

Perceived Speed and Efficiency

Another major area of analysis focused on perceived speed and efficiency. Participants were asked which system helped them locate information more quickly. The results showed that

52.9% of responses favored traditional search systems, while 23.5% preferred AI-enhanced search. The remaining responses indicated that both systems were equally effective.

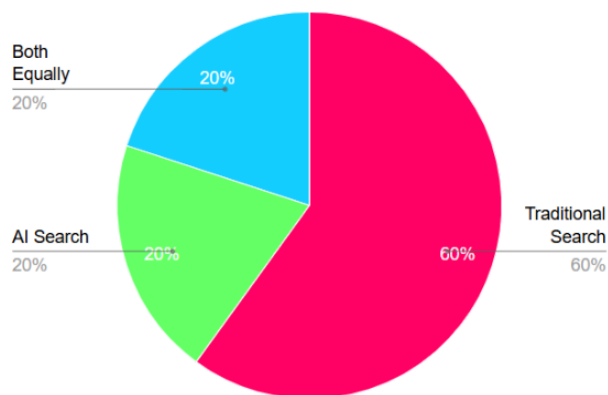
At first glance, these results may appear unexpected. AI-enhanced systems are specifically designed to provide direct answers that reduce the need to navigate multiple web pages.

However, the findings highlight an important distinction between objective response speed and perceived efficiency.

Objectively, AI systems are often capable of generating complete answers quickly. Perceived speed, however, depends on how rapidly users can process, trust, and act on the information presented. Many responses suggested that traditional search feels faster because users are already familiar with its structure. Over time, users develop efficient habits for scanning titles, identifying keywords, and selecting relevant links. This familiarity creates a smoother interaction process.

By comparison, AI-generated responses often require users to read and evaluate a larger block of synthesized text before determining whether the information is reliable. If the underlying sources are not immediately visible or easily verifiable, users may hesitate before trusting the response. As a result, even though AI systems can generate answers rapidly, the overall experience may still feel slower from the user's perspective.

25. Which system helped you find information faster? (Human Responses)



25. Which system helped you find information faster? (AI Responses)

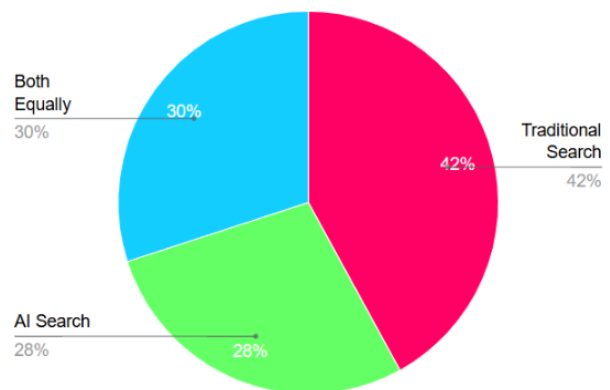


Figure 2: Human and AI Responses of speed of information retrieval of traditional vs AI enhanced search engines.

Preference Across Different Query Types

Although traditional search systems were preferred overall, this preference changed depending on the type of query being performed. When participants were asked which system they preferred for quick, everyday questions, the response distribution shifted noticeably.

Approximately 41.2% of responses favored AI-enhanced search, while 29.4% preferred traditional search and 29.4% selected both equally. Figure 3 illustrates this distribution.

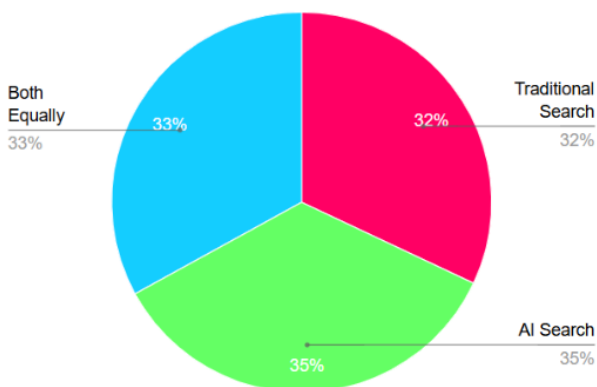
These results suggest that AI systems perform more favorably in situations where users prioritize speed and convenience over deeper exploration. For factual questions, definitions, or general

explanations, AI-generated responses provide a streamlined experience by summarizing information directly rather than requiring users to synthesize multiple sources themselves.

However, for more detailed or complex information needs, participants generally shifted back toward traditional search systems. In these cases, users valued the ability to compare sources, explore multiple perspectives, and independently verify information. This pattern indicates that search effectiveness depends heavily on the user's intent and the nature of the query itself.

Rather than pointing toward a single dominant search paradigm, these findings suggest that users adapt their search behavior depending on the task. Traditional and AI-enhanced systems appear to serve different informational purposes, with users selecting between them contextually.

7. Which system would you prefer for quick everyday questions?
(Human Responses)



7. Which system would you prefer for quick everyday questions? (AI Responses)

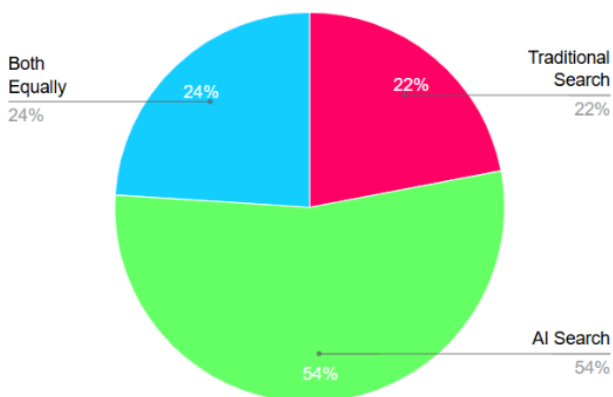


Figure 3: Human and AI Responses of preference for answering quick everyday questions of traditional vs AI enhanced search engines.

Preference for Everyday Queries

When evaluating which system is preferred for quick, everyday questions, responses were more evenly distributed. Approximately:

41.2% preferred AI systems

29.4% preferred traditional search

29.4% selected both equally

This represents a clear shift compared to overall preference results. While traditional search dominates in general use, AI systems perform more strongly in contexts where users seek quick answers rather than in-depth exploration.

This pattern indicates that AI systems are perceived as more effective for low-effort, high-speed interactions, such as simple factual queries or summarization tasks.

5.3 Human vs AI Response Patterns

A key component of the survey analysis was the comparison between human-generated and LLM-generated responses.

Human Responses

Human participants demonstrated a strong and consistent preference toward traditional search systems. Their responses were more decisive and less evenly distributed across answer choices.

Several behavioral patterns emerged:

Preference for familiar workflows

Emphasis on the ability to verify sources independently

Trust shaped by prior experience with traditional search engines

These responses suggest that human evaluations are influenced not only by system output, but also by long-term interaction patterns and expectations.

AI-Generated Responses

In contrast, LLM-generated responses were more evenly distributed across answer choices.

These responses showed a higher likelihood of selecting “AI” or “both equally,” particularly in questions involving speed or convenience.

AI responses also tended to:

Recognize strengths of both systems

Provide more balanced evaluations

Emphasize feature-based comparisons rather than experiential factors

This indicates that AI-generated evaluations are less influenced by habit or familiarity and instead reflect a more abstract comparison of system capabilities.

5.5 Summary of Findings

The results of this study highlight several key observations:

Traditional search systems remain the preferred choice overall, particularly for tasks requiring trust and verification

AI-enhanced systems perform strongly in quick-answer scenarios and everyday queries

Retrieval metrics reveal variability based on query structure and dataset limitations

Human and AI evaluations differ significantly, reflecting behavioral versus feature-based perspectives. Taken together, these findings suggest that the evolution of search is not a simple transition from traditional systems to AI-driven systems. Instead, it represents a shift toward a hybrid model, where different approaches are better suited to different types of information needs.

6. Challenges, and Debugging

A major goal of this thesis was to document the process of building and evaluating a proof-of-concept search system, not simply to present a final polished product. Because of this, the challenges encountered during implementation became an important part of the research itself. Search systems are often presented as clean pipelines: load documents, index them, retrieve results, evaluate relevance, and display outputs. In practice, each of these stages introduced complications that affected both system behavior and interpretation of results.

This section discusses the major challenges encountered during the project, including deduplication problems, mismatches between retrieved documents and benchmark relevance labels, missing relevant documents, inconsistencies in LLM-generated answers, and performance limitations. These issues shaped the final system and provided insight into the practical difficulty of evaluating modern search tools.

6.1 Deduplication and Repeated Results

One early challenge involved duplicated or near-duplicated search results. Because the system used Wikipedia-based documents from the BEIR Natural Questions dataset, multiple documents sometimes appeared with highly similar titles, overlapping content, or near-identical subject matter. From a user interface perspective, this created a poor experience because the top results could appear repetitive rather than diverse.

To address this, the system included a deduplication step when displaying sources. Retrieved documents were converted into Wikipedia-style URLs, and the interface avoided showing the

same source link more than once. This improved readability and made the output look more like a modern search engine results page.

However, this introduced an important evaluation issue. A document that appears redundant to the user may still have a distinct document ID in the benchmark dataset. Since BEIR relevance judgments are tied to document IDs rather than visual similarity, removing or hiding duplicates can change how results appear without changing how metrics are calculated. This revealed a tension between interface quality and formal evaluation.

In other words, the system had two different notions of “duplicate.” The user-facing interface treated repeated links or repeated topics as duplicates. The benchmark evaluation treated documents as distinct as long as they had different IDs. This distinction matters because improving the user experience does not always improve metric scores, and improving metric scores does not always improve the user experience.

This became one of the clearest examples of the gap between technical evaluation and perceived usefulness. Given this, in the final implementation, we removed the deduplication property, hindering visual alignment with ‘traditional’ search engines.

6.2 Retrieval Versus Benchmark Relevance

Another major challenge was the mismatch between retrieved documents that seemed useful and documents officially labeled as relevant in the qrels file. In several cases, the system retrieved

documents that were topically related to the query, but because those documents were not marked as relevant by the benchmark, the metrics treated them as incorrect.

This issue was especially important because the project relied on benchmark relevance judgments to compute Precision@5, Recall@5, and nDCG@5. These metrics assume that the qrels represent the correct relevance set. However, in practice, relevance can be broader and more subjective than the benchmark labels suggest.

For example, a query may ask about a person, event, or topic, and the system may retrieve a general article about that topic. A human evaluator might find that article useful, but if the qrels identify only a specific document as relevant, the retrieved article receives no credit. This creates a situation where the system may appear to fail according to quantitative metrics even when it provides information that is useful in practice.

This challenge revealed an important limitation of retrieval evaluation: relevance judgments are not the same as user satisfaction. Benchmark metrics are necessary for controlled comparison, but they cannot fully capture whether a user would actually find the retrieved material helpful.

6.3 The “Good Doctor” Debugging Case

One of the clearest examples of the retrieval-relevance mismatch occurred with the query:

“who is the actor that plays the good doctor on tv”

The expected answer is Freddie Highmore. In the BEIR relevance judgments, the relevant documents were specific document IDs associated with that answer. However, during testing, the system often retrieved documents related to The Good Doctor television series rather than documents specifically centered on Freddie Highmore.

At first glance, these results were not unreasonable. The retrieved documents were clearly related to the query topic. A page about The Good Doctor could contain information about the actor, cast, or character. From a human search perspective, such a result might still be useful. However, because the qrels expected specific relevant document IDs, these retrieved results were counted as non-relevant.

This case exposed the difference between answer relevance and document relevance. The user's actual information need was to identify an actor. The benchmark judged relevance at the document level. A document about the show may be topically relevant, but if it does not match the expected document ID or does not directly satisfy the answer need, the system receives no metric credit.

This debugging case also showed the difficulty of using dense retrieval for entity-specific questions. Embedding-based retrieval can successfully identify the general semantic topic of a query, but it may not always retrieve the exact document needed to answer a specific fact-based question. In this case, the system understood that the query was about The Good Doctor, but it did not consistently prioritize the page most directly associated with the actor.

The case became useful because it demonstrated that a retrieval system can be “close” in a semantic sense while still failing in a benchmark sense. This distinction became central to interpreting the project’s results.

6.6 LLM Inconsistencies in AI Overview Generation

Another challenge emerged from the AI overview component. The system used a local LLM to generate structured answers based on the top retrieved documents. The prompt instructed the model to use only the provided documents, produce a concise answer, and include light citations.

Despite this constraint, the generated answers were not always consistent. Sometimes the model produced strong summaries that accurately reflected the retrieved sources. Other times, it overgeneralized, omitted uncertainty, or presented information more confidently than the retrieved documents justified.

This introduced a second layer of possible failure. The retrieval system could fail by returning weak documents, but the LLM could also fail by misrepresenting good documents. These are different problems. If the retrieved context is poor, the generated answer may be poor because the evidence is poor. If the retrieved context is strong but the model summarizes it incorrectly, the failure belongs to the generation stage.

This distinction made debugging more difficult. When an AI overview was incorrect, it was necessary to ask:

Did the retriever return the wrong documents?

Did the correct document appear too low in the ranking?

Did the LLM ignore or misread the provided context?

Did the prompt fail to constrain the model strongly enough?

This showed why AI-integrated search systems are harder to evaluate than traditional ranked lists. In a traditional system, each result can be inspected directly. In a generative system, the final answer is shaped by both retrieval and generation, making errors harder to trace.

6.7 Prompting and Citation Challenges

The AI overview prompt included instructions to use light inline citations such as [1] and [2].

This was intended to make the generated answer resemble modern AI search summaries that cite sources.

However, citation behavior was not always reliable. The model sometimes cited sources correctly, but other times it used citations loosely or failed to connect a specific claim to the correct source. This created a traceability problem similar to the one discussed in the literature review.

This was significant because citations can create the appearance of reliability without guaranteeing faithful grounding. A generated answer with citations may look trustworthy, but unless the citations accurately support the claims being made, the user may be misled.

This challenge reinforced one of the central concerns of the thesis: AI-generated search results can improve usability, but they can also obscure the relationship between answer and evidence. Traditional search results require the user to inspect sources directly. AI overviews reduce that burden, but they also increase dependence on the system's ability to summarize and cite accurately.

6.10 Lessons Learned from Debugging

The debugging process revealed several major lessons.

First, search quality is not determined by retrieval alone. It depends on how documents are represented, how results are ranked, how duplicates are handled, how outputs are displayed, and how evaluation is defined.

Second, benchmark relevance and user usefulness are related but not identical. A system can return useful information and still receive poor metric scores if its results do not match the benchmark labels.

Third, AI integration creates new failure modes. Traditional systems usually fail by missing or misranking documents. AI systems can also fail by generating unsupported, overconfident, or poorly cited answers.

Fourth, performance tradeoffs are central to system design. Dense embeddings and LLM summaries add semantic power, but they also introduce indexing cost, loading time, and generation latency.

Finally, the failures were not separate from the research. They became evidence. Each debugging issue clarified a tradeoff between traditional retrieval and AI-enhanced search. Because this thesis is a proof-of-concept exploration, these challenges are not simply obstacles. They are part of the project's contribution.

6.11 Summary

The implementation and evaluation process exposed the practical complexity of building a search system. Deduplication improved the interface but complicated evaluation. Benchmark relevance helped measure retrieval but did not always match human-perceived usefulness. Dense retrieval captured semantic similarity but sometimes missed exact answer targets.

LLM-generated summaries improved presentation but introduced inconsistency and traceability problems. Performance constraints required separating indexing from retrieval and caching expensive resources.

These challenges demonstrate that search is not a single problem with a single solution. It is a layered system of tradeoffs. The custom search engine made these tradeoffs visible and provided a concrete basis for comparing traditional retrieval, neural retrieval, and generative AI search.

7. Discussion

The results of this study highlight a fundamental shift in how search systems operate and how users interact with them. While traditional information retrieval and AI-enhanced search systems are often framed as competing paradigms, the findings suggest that they are better understood as complementary approaches with distinct strengths and tradeoffs. This section interprets those tradeoffs through several key dimensions: retrieval versus generation, transparency versus convenience, perceived effectiveness, and task-specific performance.

7.1 Retrieval vs. Generation

One of the most important distinctions revealed by this study is the difference between retrieval-based systems and generation-based systems.

Traditional search engines operate by retrieving and ranking documents. Their output is a structured list of sources, leaving the task of synthesis to the user. This approach prioritizes

completeness and traceability. Every result corresponds to a document that can be inspected, verified, and interpreted independently.

In contrast, AI-enhanced systems increasingly rely on generation, producing synthesized answers that combine information from multiple sources. This reduces the effort required from the user, as the system performs part of the reasoning process.

However, this shift introduces a tradeoff. Retrieval systems may require more effort to navigate, but they preserve the connection between information and its source. Generative systems provide immediate answers, but they abstract away the underlying evidence. As observed during implementation and debugging, this abstraction can introduce ambiguity when errors occur. If a generated answer is incorrect, it is not always clear whether the issue lies in retrieval, ranking, or summarization.

The results suggest that generation improves usability but complicates evaluation and reliability. Retrieval provides structure and traceability, while generation provides convenience and synthesis.

7.2 Transparency vs. Convenience

Closely related to the retrieval-generation distinction is the tradeoff between transparency and convenience. Traditional search systems are relatively transparent. Although their internal algorithms may be complex, the output itself is interpretable. Users can see multiple sources,

compare perspectives, and verify claims. This supports a more active form of information consumption.

AI-enhanced systems, on the other hand, prioritize convenience. By presenting a single summarized answer, they reduce the need for users to open multiple links and synthesize information manually. This aligns with modern expectations for speed and efficiency. However, this convenience comes at the cost of transparency. As noted in both the literature and implementation process, generative outputs often lack clear traceability. Even when citations are provided, the connection between specific claims and specific sources may not be fully reliable. This creates a situation where users must trust the system's interpretation rather than directly evaluating the underlying evidence.

The survey results reflect this tension. While AI systems performed well in scenarios involving quick answers, traditional systems remained dominant in overall preference. This suggests that users value convenience, but not at the expense of confidence and control.

7.3 Perceived Effectiveness vs. Measured Performance

Another key insight from this study is the gap between measured retrieval performance and perceived usefulness.

Information retrieval metrics such as Precision@k and nDCG@k provide structured ways to evaluate system performance. However, these metrics depend on predefined relevance judgments and do not fully capture how users experience search results.

The survey data showed that users often preferred traditional search systems, even in cases where AI systems provided faster or more concise answers. This indicates that perceived effectiveness is influenced by factors beyond raw retrieval quality, including familiarity, trust, and the ability to verify information.

At the same time, AI systems were favored for everyday queries and quick information needs. This suggests that users evaluate systems differently depending on context. A system that is perceived as “better” overall may not be the one that is chosen for a specific task.

This divergence highlights a limitation of relying solely on quantitative metrics. A system can perform well according to formal evaluation but still feel less useful to users, or vice versa. Effective evaluation must therefore consider both technical performance and user perception.

7.4 Why AI Systems “Feel” Better

Despite the continued dominance of traditional search in overall preference, AI systems often feel more effective in specific situations. This can be explained by several factors.

First, AI-generated answers reduce cognitive load. Instead of requiring users to scan multiple documents, the system presents a synthesized response. This creates the impression of efficiency, even if the underlying retrieval process is similar.

Second, AI systems provide immediate closure. Traditional search presents options, while AI systems present conclusions. For many users, especially in casual contexts, this aligns more closely with their expectations of what a search system should do.

Third, AI systems are designed to produce coherent and conversational output, which can feel more natural than navigating a list of links. This stylistic difference contributes to the perception that AI systems are more advanced, even when their underlying retrieval performance is not significantly better.

However, this perceived advantage can be misleading. As observed during debugging, AI-generated answers may omit uncertainty, overgeneralize, or present information more confidently than the evidence supports. This creates a tradeoff between clarity and accuracy.

7.5 Where Traditional Systems Still Excel

The results of this study indicate that traditional search systems remain highly effective in several key areas.

First, they provide strong support for verification and exploration. Users can access multiple sources, compare information, and form their own conclusions. This is especially important for academic, technical, or high-stakes queries.

Second, traditional systems offer consistent and predictable behavior. Given the same query and index, they produce stable results that can be evaluated systematically. This makes them easier to debug and analyze.

Third, they align more closely with established evaluation frameworks. Metrics such as precision and recall are designed for document retrieval, making traditional systems easier to assess using standard benchmarks.

Finally, traditional systems support open-ended information seeking. Rather than guiding users toward a single answer, they provide a space for exploration. This can be valuable in research contexts where the goal is not just to find an answer, but to understand a topic more broadly.

7.6 Toward a Hybrid Model of Search

Taken together, the findings suggest that the future of search is not a replacement of traditional systems by AI, but a shift toward hybrid systems that combine retrieval and generation.

In such systems, retrieval provides the underlying structure and evidence, while generation provides synthesis and usability. The challenge is to balance these components in a way that preserves transparency while improving efficiency.

The implementation and evaluation in this thesis demonstrate that both components have value, but also introduce distinct challenges. Designing effective search systems will require careful consideration of how these elements interact, particularly in terms of trust, evaluation, and user experience.

8. Conclusion

This thesis explored the evolution of search systems through the construction and evaluation of a custom retrieval-based search engine and its comparison with AI-enhanced search platforms. Rather than attempting to determine a definitive “better” system, the goal was to examine how different architectures behave in practice and what tradeoffs they introduce.

The implementation process provided insight into the mechanics of traditional search systems, including indexing, retrieval, ranking, and evaluation. It also revealed the challenges involved in building and testing such systems, from handling duplicate results to interpreting benchmark relevance judgments.

The evaluation results showed that traditional search systems remain strongly preferred overall, particularly in contexts requiring trust, verification, and structured exploration. At the same time, AI-enhanced systems demonstrated advantages in speed, summarization, and ease of use, especially for everyday queries.

A key finding of this study is the gap between measured performance and perceived usefulness. Retrieval metrics provide valuable quantitative insights, but they do not fully capture how users experience search systems. User perception is influenced by familiarity, trust, and interaction patterns, which can lead to preferences that differ from metric-based evaluations.

Ultimately, this thesis suggests that AI search is not replacing traditional search, but augmenting it in specific contexts. Each approach offers strengths that are suited to different types of information needs.

Future Work

There are several directions for future research.

First, obtaining IRB approval would allow for a larger and more diverse set of human participants, improving the validity of the survey-based evaluation.

Second, expanding the system to include both lexical and neural retrieval methods (e.g., combining BM25 with embeddings) would allow for deeper analysis of hybrid retrieval performance.

Third, maintaining a consistent set of questions between all 3 components of the survey (Sarang, myself and AI responses) can allow us to more effectively evaluate which is ‘better’ by reducing variable differences.

Finally, future work could explore more robust evaluation frameworks for generative search systems, as traditional retrieval metrics do not fully capture the quality of synthesized answers.

Bibliography

- Brin, S., & Page, L. (1998). The anatomy of a large-scale hypertextual Web search engine. *Computer Networks and ISDN Systems*, 30(1–7), 107–117.
<http://infolab.stanford.edu/pub/papers/google.pdf>
- Hersh, W. (2024). *Search still matters: Information retrieval in the era of generative AI*. *Journal of the American Medical Informatics Association*, 31(9), 2159–2162. Retrieved from <https://pmc.ncbi.nlm.nih.gov/articles/PMC11339511/>
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
<https://nlp.stanford.edu/IR-book/pdf/irbookonlinereading.pdf>
- Mitra, B., & Craswell, N. (2017). An introduction to neural information retrieval. *Foundations and Trends® in Information Retrieval*, 13(1), 1–126.
<https://www.microsoft.com/en-us/research/wp-content/uploads/2017/06/INR-061-Mitra-neuralir-intro.pdf>
- Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework, BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
https://www.staff.city.ac.uk/~sbrp622/papers/foundations_bm25_review.pdf
- Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.
<https://papers.nips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>